

jamie-lemon

Docs: removes module.rst

53c9aaf · 5 days ago



 .github	.github/ISSUE_TEMPLATE/bug_repo...	last week
 .vs	Update VSWorkspaceState.json	4 years ago
 docs	Docs: removes module.rst	5 days ago
 scripts	scripts/test.py: use autoenv.py.	2 months ago
 src	src/__init__.py:JM_get_font(): cope ...	last week
 src_classic	adjust to python 3.14	last year
 tests	tests/test_general.py:test_5054(): e...	last week
 .gitattributes	.gitattributes: new, mark .pdf files ar...	2 years ago
 .gitignore	Initial commit for Korean version of ...	9 months ago
 .python-version	.python-version: new, needed for Git...	2 years ago
 .readthedocs.yaml	Updates the Ubuntu OS in the .yaml ...	2 months ago
 COPYING	Cleanup license information	5 years ago
 README.md	README.md: updated example conv...	2 months ago
 changes.txt	changes.txt: updates for 1.28.2.	last week
 pyproject.toml	pyproject.toml: require package pipc...	4 months ago
 pytest.ini	pytest.ini: use -tb=native to reduce ...	6 months ago
 setup.py	setup.py: increment version to 1.28.2.	last week
 valgrind.supp	valgrind.supp: new, valgrind exclusio...	3 years ago

README

AGPL-3.0 license





PyMuPDF



The PDF engine behind over 50 million monthly downloads, powering AI pipelines worldwide.

PyMuPDF is a high-performance Python library for data extraction, analysis, conversion, rendering and manipulation of PDF (and other) documents. Built on top of MuPDF — a lightweight, fast C engine — PyMuPDF gives you precise, low-level control over documents alongside high-level convenience APIs. No mandatory external dependencies.



Why PyMuPDF?

- **Fast** — powered by [MuPDF](#), a best-in-class C rendering engine
- **Accurate** — pixel-perfect text extraction with font, color, and position metadata
- **Versatile** — read, write, annotate, redact, merge, split, and convert documents
- **LLM-ready** — native Markdown output via [PyMuPDF4LLM](#) for RAG and AI pipelines
- **No mandatory dependencies** — `pip install pymupdf` and you're done

Installation

```
pip install pymupdf
```



Wheels are available for **Windows**, **macOS**, and **Linux** on Python 3.10–3.14. If no pre-built wheel exists for your platform, pip will compile from source (requires a C/C++ toolchain).

Optional extras

Package	Purpose
pymupdf-fonts	Extended font collection for text output

Package	Purpose
pymupdf4llm	LLM/RAG-optimised Markdown and JSON extraction
pymupdfpro	Adds Office document support
tesseract-ocr	OCR for scanned pages and images (separate install)

```
# More fonts
pip install pymupdf-fonts

# LLM-ready extraction
pip install pymupdf4llm

# Office support
pip install pymupdfpro

# OCR (Tesseract must be installed separately)
# macOS
brew install tesseract

# Ubuntu / Debian
sudo apt install tesseract-ocr
```



Supported File Formats

Input

Category	Formats
PDF & derivatives	PDF, XPS, EPUB, CBZ, MOBI, FB2, SVG, TXT, MD
Images	PNG, JPEG, BMP, TIFF, GIF, and more
Microsoft Office (<i>Pro</i>)	DOC, DOCX, XLS, XLSX, PPT, PPTX
Korean Office (<i>Pro</i>)	HWP, HWPX

Output

Format	Notes
PDF	Full fidelity conversion from Office formats
SVG	Vector page rendering
Image (PNG, JPEG, ...)	Page rasterisation at any DPI
Markdown	Structure-aware, LLM-ready
JSON	Bounding boxes, layout data, per-element detail
Plain text	Fast, lightweight extraction

[🔗 Quick start](#)

[🔗 Extract text](#)

```
import pymupdf

doc = pymupdf.open("document.pdf")
for page in doc:
    print(page.get_text())
```



[🔗 Extract text with layout metadata](#)

```
import pymupdf

doc = pymupdf.open("document.pdf")
page = doc[0]

blocks = page.get_text("dict")["blocks"]
for block in blocks:
    if block["type"] == 0: # text block
        for line in block["lines"]:
            for span in line["spans"]:
                print(f"{span['text']}!r}  font={span['font']}  size={span['size']:.1f}")
```



[🔗 Extract tables](#)

```
import pymupdf

doc = pymupdf.open("spreadsheet.pdf")
page = doc[0]

tables = page.find_tables()
for table in tables:
    print(table.to_markdown())

# or get as Pandas DataFrame
df = table.to_pandas()
```



[🔗 Render a page to an image](#)

```
import pymupdf

doc = pymupdf.open("document.pdf")
page = doc[0]

pixmap = page.get_pixmap(dpi=150)
pixmap.save("page_0.png")
```



🔗 OCR a scanned document

```
import pymupdf

doc = pymupdf.open("scanned.pdf")
page = doc[0]

# Requires Tesseract installed and on PATH
text = page.get_textpage_ocr(language="eng").extractText()
print(text)
```



🔗 Convert Markdown to PDF

```
import pymupdf

md_doc = pymupdf.open("example.md")
md_doc.save("example.pdf")
```



🔗 Convert to Markdown for LLMs

```
import pymupdf4llm

md = pymupdf4llm.to_markdown("report.pdf")
# Pass directly to your LLM or vector store
print(md)
```



🔗 Annotate and redact

```
import pymupdf

doc = pymupdf.open("contract.pdf")
page = doc[0]

# Add a highlight annotation
rect = pymupdf.Rect(72, 100, 400, 120)
page.add_highlight_annot(rect)

# Add a redaction and apply it
page.add_redact_annot(rect)
page.apply_redactions()

doc.save("contract_redacted.pdf")
```



🔗 Merge PDFs

```
import pymupdf

merger = pymupdf.open()
for path in ["part1.pdf", "part2.pdf", "part3.pdf"]:
    merger.insert_pdf(pymupdf.open(path))
```



```
merger.save("merged.pdf")
```

🔗 Convert an Office document to PDF

```
import pymupdf.pro

pymupdf.pro.unlock("YOUR-LICENSE-KEY")

doc = pymupdf.open("presentation.pptx")
pdf_bytes = doc.convert_to_pdf()

with open("output.pdf", "wb") as f:
    f.write(pdf_bytes)
```



🔗 Extract LLM-ready Markdown from a Word document

```
import pymupdf4llm
import pymupdf.pro

pymupdf.pro.unlock("YOUR-LICENSE-KEY")

md = pymupdf4llm.to_markdown("document.docx")
print(md)
```



🔗 Features

🔗 Core capabilities

Feature	Description
Text extraction	Plain text, rich dict (font, size, color, bbox), HTML, XML, raw blocks
Table detection	<code>find_tables()</code> — locate, extract, and export tables as Markdown or structured data
Image extraction	Extract embedded images and render any page to a high-resolution <code>Pixmap</code>
Rendering	Render PDF pages to images or <code>Pixmap</code> data for use in UI or other workflows
OCR	Tesseract integration — full-page or partial OCR, configurable language
Annotations	Read and write highlights, underlines, squiggly lines, sticky notes, free text, ink, stamps
Redaction	Add and permanently apply redaction annotations
Forms	Read and fill PDF AcroForm fields
PDF creation	Create PDFs directly with the API or quickly convert from Markdown files
PDF editing	Insert, delete, and reorder pages; set metadata; merge and split documents

Feature	Description
Drawing	Draw lines, curves, rectangles, and circles; insert HTML boxes
Encryption	Open password-protected PDFs; save with RC4 or AES encryption
Links	Extract hyperlinks, internal cross-references, and URI targets
Bookmarks	Read and write the outline / table of contents tree
Metadata	Title, author, creation date, producer, subject, and custom entries
Color spaces	RGB, CMYK, greyscale; color space conversion

[🔗 LLM & AI output \(via PyMuPDF4LLM\)](#)

Output	API
Markdown	<code>pymupdf4llm.to_markdown(path)</code>
JSON	<code>pymupdf4llm.to_json(path)</code>
Plain text	<code>pymupdf4llm.to_text(path)</code>

Supports multi-column layouts, natural reading order and page chunking.

 DEMO

PYMUPDF4LLM

[🔗 Supported Python versions](#)

Python **3.10 – 3.14** (as of v1.27.x). Wheels ship for:

- manylinux x86_64 and aarch64
- musllinux x86_64
- macOS x86_64 and arm64
- Windows x86 and x86_64

[🔗 Performance](#)

PyMuPDF is built on MuPDF — one of the fastest PDF rendering engines available. Typical benchmarks against pure-Python PDF libraries show **10–50× speed improvements** for text extraction and **100× or more** for page rendering, with a minimal memory footprint.

For AI workloads, PyMuPDF4LLM processes documents **without a GPU**, cutting infrastructure costs significantly compared to vision-based LLM approaches.

🔗 Recipes

- ▶ Extract all images from a PDF
- ▶ Search for text across a document
- ▶ Split a PDF into individual pages
- ▶ Insert a watermark on every page

🔗 Office Document Processing

PyMuPDF can be extended with PyMuPDF Pro. This adds a conversion layer that handles Microsoft and Korean Office formats natively – no Office installation, no COM interop, no LibreOffice subprocess.

Once unlocked, `pymupdf.open()` accepts Office files exactly like PDFs:

```
import pymupdf.pro
pymupdf.pro.unlock("YOUR-LICENSE-KEY")

# Works identically regardless of format
for fmt in ["contract.docx", "data.xlsx", "deck.pptx", "report.hwpx"]:
    doc = pymupdf.open(fmt)
    for page in doc:
        print(page.get_text())
```



[Get a trial license key for PyMuPDF Pro](#)

What you can do with Office documents:

- Extract text and images page-by-page
- Convert to PDF with `doc.convert_to_pdf()`
- Rasterise pages to PNG/JPEG for visual inspection
- Feed directly into PyMuPDF4LLM for AI-ready output

🔗 Restrictions Without a License Key

When `pymupdf.pro.unlock()` is called **without** a key, the following restrictions apply:

Restriction	Detail
Page limit	Only the first 3 pages of any document are accessible
Time limit	Evaluation period – functionality expires after a set duration

All other Pro features work normally within these constraints, making it straightforward to prototype before purchasing a license.

🔗 Frequently Asked Questions

🔗 Can I use PyMuPDF, PyMuPDF4LLM and PyMuPDF Pro without sending data to the cloud?

Yes, absolutely — and this is one of PyMuPDF's most significant advantages.

PyMuPDF runs entirely locally. It is a native Python library built on top of the MuPDF C engine. When you call `pymupdf.open()`, `page.get_text()`, `page.find_tables()`, or any other method, everything executes in-process on your own machine. No data is transmitted anywhere.

There are no telemetry calls, no licence validation callbacks, no cloud dependencies of any kind in the open-source AGPL build or the commercial build. Once the package is installed, it works fully air-gapped.

This makes PyMuPDF well-suited for:

- Regulated industries — healthcare (HIPAA), finance, legal, government, where documents cannot leave a controlled environment
- On-premise deployments — servers with no outbound internet access
- Air-gapped systems — classified or sensitive environments
- Self-hosted RAG pipelines — processing confidential documents locally before feeding an on-premise LLM
- Saving on token costs for document pre-processing before sending data to your LLM

The only thing you need an internet connection for is the initial `pip install`. After that, the package and all its capabilities are entirely self-contained.

🔗 Should I `import pymupdf` or `import fitz`?

Use `import pymupdf`. The `fitz` name is a legacy alias that still works as of v1.24.0+, but `import pymupdf` is the recommended and future-proof approach. The two are interchangeable in existing code:

```
import pymupdf          # recommended
# import fitz           # legacy alias — still works but avoid for new code
```



🔗 Does PyMuPDF work with Korean, Japanese, or Chinese documents?

Yes — PyMuPDF has solid CJK support

🔗 How do I extract Markdown from PDF for LLM?

Let PyMuPDF4LLM do everything (recommended for RAG).

PyMuPDF4LLM is a high-level wrapper that outputs standard text and table content together in an integrated Markdown-formatted string across all document pages PyMuPDF — tables are detected, converted to GitHub-compatible Markdown, and interleaved with surrounding text in the correct reading order. This is the best starting point for feeding an LLM or building a RAG pipeline.

```
import pymupdf4llm
```



```
md = pymupdf4llm.to_markdown("report.pdf")
print(md)
# Tables appear as Markdown | col1 | col2 | ... inline with the text
```

🔗 Text extraction returns garbled characters or empty output. Why?

This usually means the PDF uses custom font encodings without a proper character map (CMAP). The font's glyphs are present but cannot be mapped back to Unicode. In these cases:

- Use OCR as a fallback (`page.get_textpage_ocr()`)
- Consider that scanned PDFs will always need OCR – text extraction on scans returns nothing

🔗 How do I extract text from a specific area of a page?

Pass a `clip` rectangle to `get_text()`:

```
import pymupdf

doc = pymupdf.open("input.pdf")
page = doc[0]

# Define the area you want (x0, y0, x1, y1) in points
clip = pymupdf.Rect(50, 100, 400, 300)
text = page.get_text("text", clip=clip)
```



🔗 How do I search for text and find its location on the page?

```
import pymupdf

doc = pymupdf.open("input.pdf")
page = doc[0]

# Returns a list of Rect objects surrounding each match
locations = page.search_for("invoice number")
for rect in locations:
    print(rect) # e.g. Rect(72.0, 120.5, 210.0, 134.0)
```



🔗 `get_images` shows no images but I can clearly see charts in the PDF. Why?

Charts and diagrams created by tools like matplotlib, Excel, or R are typically rendered as vector graphics (PDF drawing commands), not raster images. `get_images` only lists embedded raster image objects and will not detect vector graphics. To capture these, rasterise the entire page with `page.get_pixmap()`.

🔗 How does OCR work in PyMuPDF? Does it require a separate Tesseract installation?

PyMuPDF uses MuPDF's built-in Tesseract-based OCR support, so there is no Python-level `pytesseract` dependency. However, PyMuPDF still needs access to the **Tesseract language data files** (`tessdata`), and automatic `tessdata` discovery may invoke the `tesseract` executable (for example, to list available languages) if you do not explicitly provide a `tessdata` path. In practice, the recommended setup is to either install Tesseract so discovery works automatically, or configure the `tessdata` location yourself via the `tessdata` parameter or the `TESSDATA_PREFIX` environment variable. Over 100 languages are supported.

```
import pymupdf

doc = pymupdf.open("scanned.pdf")
page = doc[0]

# Get a text page using OCR
tp = page.get_textpage_ocr(language="eng")
text = page.get_text(textpage=tp)
print(text)
```



🔗 How do I run OCR on a standalone image file (not a PDF)?

```
import pymupdf

pix = pymupdf.Pixmap("image.png")
if pix.alpha:
    pix = pymupdf.Pixmap(pix, 0) # remove alpha channel – required for OCR

# Wrap in a 1-page PDF and OCR it
doc = pymupdf.open()
page = doc.new_page(width=pix.width, height=pix.height)
page.insert_image(page.rect, pixmap=pix)
tp = page.get_textpage_ocr()
text = page.get_text(textpage=tp)
```



🔗 How do I highlight text in a PDF?

```
import pymupdf

doc = pymupdf.open("input.pdf")
page = doc[0]

# Use quads=True for accurate highlights on non-horizontal text
quads = page.search_for("important term", quads=True)
page.add_highlight_annot(quads)

doc.save("highlighted.pdf")
```



PyMuPDF supports all standard PDF text markers: highlight, underline, strikeout, and squiggly.

🔗 How do I permanently redact (remove) content from a PDF?

Redaction is a deliberate two-step process so you can review before committing:

```
import pymupdf

doc = pymupdf.open("input.pdf")
page = doc[0]

# Step 1: Mark the area(s) to redact
rect = page.search_for("confidential")[0]
page.add_redact_annot(rect, fill=(1, 1, 1)) # white fill

# Step 2: Apply – permanently removes the underlying content
page.apply_redactions()

doc.save("redacted.pdf")
```



After `apply_redactions()`, the original content is gone. It cannot be recovered from the saved file.

🔗 How do I read form field values from a PDF?

```
import pymupdf

doc = pymupdf.open("form.pdf")
page = doc[0]

for field in page.widgets():
    print(f"{field.field_name}: {field.field_value}")
```



🔗 How do I fill in a PDF form programmatically?

```
import pymupdf

doc = pymupdf.open("form.pdf")
page = doc[0]

for field in page.widgets():
    if field.field_name == "First Name":
        field.field_value = "Ada"
        field.update()

doc.save("filled_form.pdf")
```



🔗 Can I use multithreading with PyMuPDF?

No. PyMuPDF does not support multithreaded use, even with Python's newer free-threading mode. The underlying MuPDF library only provides partial thread safety, and a fully thread-safe PyMuPDF implementation would still impose a single-threaded overhead — negating the benefit.

Use multiprocessing instead. Each process opens the file independently and works on its own page range:

```
from multiprocessing import Pool
import pymupdf
```



```
def process_pages(args):
    path, start, end = args
    doc = pymupdf.open(path) # each process opens its own handle
    results = []
    for i in range(start, end):
        results.append(doc[i].get_text())
    return results

with Pool(4) as pool:
    chunks = [("input.pdf", 0, 25), ("input.pdf", 25, 50), ...]
    all_results = pool.map(process_pages, chunks)
```

🔗 How can I speed up repeated text extraction on the same page?

Reuse a `TextPage` object. Creating a `TextPage` is the expensive part — once created, switching between extraction formats is cheap:

```
import pymupdf

page = doc[0]
tp = page.get_textpage() # create once

text = page.get_text("text", textpage=tp)
words = page.get_text("words", textpage=tp)
data = page.get_text("dict", textpage=tp)
```



This can reduce execution time by 50–95% for repeated extractions on the same page.

🔗 How do I read and write PDF metadata?

```
import pymupdf

doc = pymupdf.open("input.pdf")

# Read
print(doc.metadata)
# {'title': '...', 'author': '...', 'subject': '...', 'keywords': '...', ...}

# Write
doc.set_metadata({
    "title": "Annual Report 2025",
    "author": "Finance Team",
    "keywords": "annual, finance, 2025"
})
doc.save("output.pdf")
```



🔗 How do I read or set the table of contents / bookmarks?

```
import pymupdf

doc = pymupdf.open("input.pdf")
```



```
# Read – returns a list of [level, title, page_number] entries
toc = doc.get_toc()
for level, title, page in toc:
    print(" " * level, title, "→ page", page)

# Write
new_toc = [
    [1, "Introduction", 1],
    [1, "Methods", 5],
    [2, "Data sources", 6],
]
doc.set_toc(new_toc)
doc.save("output.pdf")
```

 Documentation

Full installation guide, API reference, cookbook, and tutorial at pymupdf.readthedocs.io.

- [Installation guide](#)
- [API reference](#)
- [Cookbook](#)
- [Tutorial](#)
- [Changelog](#)
- [PyMuPDF4LLM docs](#)
- [PyMuPDF Pro docs](#)

 Related projects

Project	Description
PyMuPDF4LLM	LLM/RAG-optimised Markdown and JSON extraction
PyMuPDF Pro	Adds Office and HWP document support
pymupdf-fonts	Extended font collection for PyMuPDF text output

 Licensing

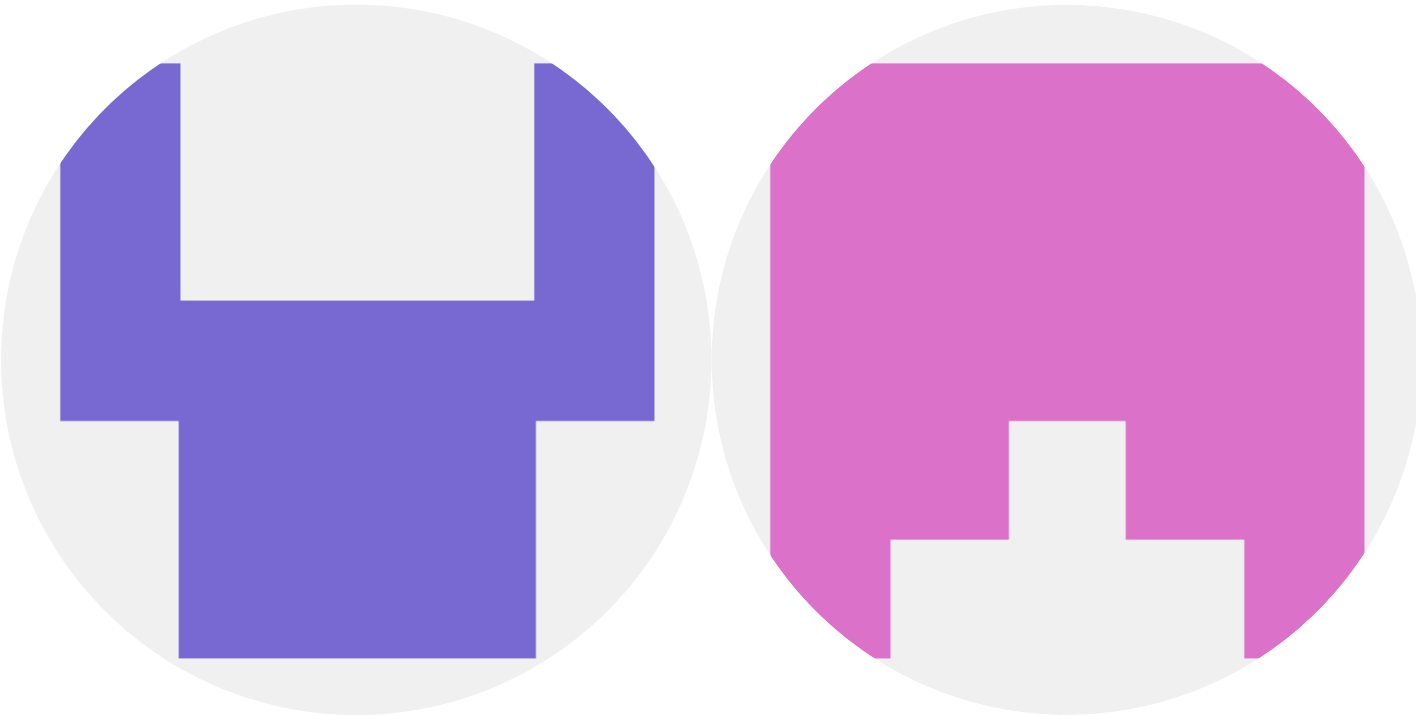
Releases 173

 **PyMuPDF-1.28.2 released** Latest

5 days ago

[+ 172 releases](#)

Used by 76K



Contributors 89



[+ 75 contributors](#)

Languages

Python 65.2% SWIG 34.1% Other 0.7%